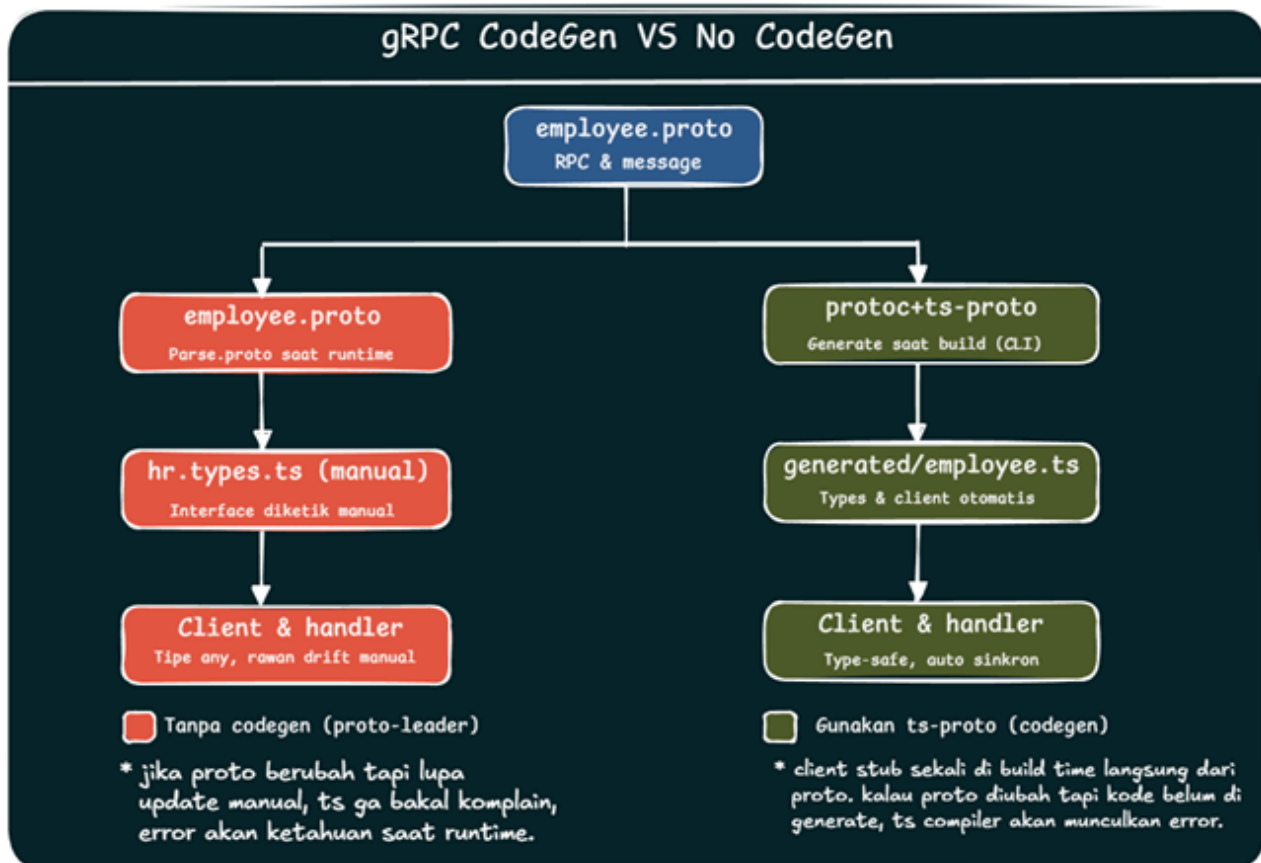


01-02. Intro gRPC

Ada 2 metode untuk gunakan proto di gRPC, kita bisa gunakan metode:

1. Manual : dimana interface kita ketik manual sendiri.
2. Codegen : kita bisa gunakan tool codegen untuk menghasilkan proto generator.



1. Install Dependency

```
npm install @grpc/grpc-js @grpc/proto-loader @grpc/reflection
npm install -D @types/google-protobuf
```

2. Setting environment

Buka file .env dan setting port untuk gRPC.

```
GRPC_PORT=50051
```

2. Definisikan .proto

```
// src/grpc/protos/employee.proto

syntax = "proto3";

package hr;

//interface
service EmployeeService {
    // Digunakan saat payroll butuh return 1 employee misal saat generate salary
    // bisa digunakan juga saat fallback consumer kafka
    rpc GetEmployee(GetEmployeeRequest) returns (EmployeeResponse);

    // Digunakan untuk generate payroll bulanan semua employee
    // diproses secara streaming
    rpc GetEmployeesForPayroll(GetEmployeesForPayrollRequest) returns (stream EmployeeResponse);
}

message GetEmployeesForPayrollRequest {
    int32 department_id = 1;
    string employment_status = 2;
}

message GetEmployeeRequest {
    int32 employee_id = 1;
}

message EmployeeResponse {
    int32 employee_id = 1;
    string first_name = 2;
    string last_name = 3;
    string email = 4;
    double salary = 5;
    int32 job_id = 6;
    int32 department_id = 7;
    string employment_status = 8;
    string employment_type = 9;
    string hire_date = 10;
    string updated_at = 11;
}
```

3. Create gRPC Handler

```
//src/modules/employees/employees.grpc-handler.ts

import type { sendUnaryData, ServerUnaryCall, ServerWritableStream } from
'@grpc/grpc-js';
import { status } from '@grpc/grpc-js';
import * as employeeService from '../employees.service';
import { ApiError } from '../../common/utils/api-error';
import type { EmployeeApiRow } from '../employees.types';

function toGrpcResponse(row: EmployeeApiRow) {
  return {
    employee_id: row.employeeId,
    first_name: row.firstName ?? '',
    last_name: row.lastName,
    email: row.email,
    salary: row.salary,
    job_id: row.jobId,
    department_id: row.departmentId ?? 0,
    employment_status: row.employmentStatus,
    employment_type: row.employmentType,
    hire_date: row.hireDate,
    updated_at: row.updatedAt,
  };
}

// Mapping ApiError (HTTP-style) -> gRPC status code
function toGrpcError(err: unknown) {
  if (err instanceof ApiError) {
    const code = err.statusCode === 404 ? status.NOT_FOUND :
status.INVALID_ARGUMENT;
    return { code, message: err.message };
  }
  return { code: status.INTERNAL, message: 'Internal error' };
}

export async function getEmployee(
  call: ServerUnaryCall<{ employee_id: number }, unknown>,
  callback: sendUnaryData<ReturnType<typeof toGrpcResponse>>,
) {
  try {
    const employee = await
employeeService.getEmployeeById(call.request.employee_id);
    callback(null, toGrpcResponse(employee));
  } catch (err) {
    callback(toGrpcError(err) as any, null);
  }
}

export async function getEmployeesForPayroll(
  call: ServerWritableStream<{ department_id: number; employment_status:
```

```

string }, unknown>,
) {
  try {
    // Streaming -> kita paging manual dari findAll yang sudah ada,
    // kita push per-batch ke stream biar ga sekali tarik 50rb row
    let page = 1;
    const limit = 15;
    const { department_id, employment_status } = call.request;

    while (true) {
      const { items, pagination } = await employeeService.listEmployees({
        page,
        limit,
        departmentId: department_id || undefined,
        employmentStatus: (employment_status || undefined) as any,
      });

      for (const item of items) call.write(toGrpcResponse(item));

      if (page >= pagination.totalPages) break;
      page++;
    }
    call.end();
  } catch (err) {
    call.emit('error', toGrpcError(err));
  }
}

```

4. Boot gRPC Server

```

// src/grpc/server.ts
import path from 'node:path';
import { Server, ServerCredentials, loadPackageDefinition } from '@grpc/grpc-js';
import { loadSync } from '@grpc/proto-loader';
import { ReflectionService } from '@grpc/reflection';
import * as employeeGrpcHandler from
'../modules/employees/grpc/employees.grpc-handler';

/**
 * Promise<Server> return promise, ini memastikan port sudah di bind (ikat),
 * jangan sampai gRPC server sudah running, tapi port belum dibinding.
 * Jika Sukses kita return resolve
 * Jika Failed kita return reject
 */
export function startGrpcServer(port = 50051): Promise<Server> {
  return new Promise((resolve, reject) => {
    const protoPath = path.join(__dirname, 'protos/employee.proto');
    const packageDef = loadSync(protoPath, {

```

```
    keepCase: true,
    longs: Number,
    enums: String,
    defaults: true,
  });

  const proto = loadPackageDefinition(packageDef) as any;
  const server = new Server();

  server.addService(proto.hr.EmployeeService.service, {
    GetEmployee: employeeGrpcHandler.getEmployee,
    GetEmployeesForPayroll: employeeGrpcHandler.getEmployeesForPayroll
  });

  /**
   * Gunakan reflection agar semua method/interface muncul di tool seperti
   * Postman/Appollo,
   * tanpa reflection kita harus tentukan path file .proto nya.
   */
  const reflection = new ReflectionService(packageDef);
  reflection.addToServer(server);

  server.bindAsync(`0.0.0.0:${port}`, ServerCredentials.createInsecure(),
    (err, boundPort) => {
      if (err) {
        reject(err);
        return;
      }
      console.log(`[grpc] listening on :${boundPort}`);
      resolve(server);
    });
  });
}
```

5. Register gRPC Server To Fastify Server

```
// src/server.ts
import { createApp } from './app';
import { env } from './config/env';
import { checkDatabaseConnection, pool } from './db';
import { startGrpcServer } from './grpc/server';

async function bootstrap(): Promise<void> {
  await checkDatabaseConnection();
  console.log(' Koneksi PostgreSQL OK');

  // REST server (Fastify)
  const app = await createApp();
  await app.listen({ port: env.PORT, host: '0.0.0.0' });
}
```

```
console.log(`REST server berjalan di http://localhost:${env.PORT}`);
console.log(`Health check : http://localhost:${env.PORT}/health`);
console.log(`API prefix : ${env.API_PREFIX}`);

//startGrpcServer(50051);
const grpcServer = await startGrpcServer(env.GRPC_PORT ?? 50051);
console.log(` gRPC server : 0.0.0.0:${env.GRPC_PORT ?? 50051}`);

const shutdown = async (signal: string) => {
  console.log(`\n${signal} diterima, mematikan server dengan aman...`);

  await app.close();
  /**
   * saat exit gunakan Ctrl + C di terminal, tryShutdown() akan dicall
   * ini dilakukan untuk memastikan semua port yang dipakai di-relase
   * tidak ada yang menggantung, kita kasih waktu 5s pake timeout fallback,
   * jika masih belum selesai kita paksa supaya di closed
   */

  await new Promise<void>((resolve) => {
    const forceTimeout = setTimeout(() => {
      console.warn('gRPC graceful shutdown timeout (5s), force
shutdown...');
      grpcServer.forceShutdown();
      resolve();
    }, 5000);

    grpcServer.tryShutdown((err) => {
      clearTimeout(forceTimeout);
      if (err) {
        console.error('gRPC graceful shutdown gagal, force shutdown:',
err);
        grpcServer.forceShutdown();
      } else {
        console.log('gRPC server ditutup.');
```

```
bootstrap().catch((err) => {  
  console.error('Gagal start aplikasi:', err);  
  process.exit(1);  
});
```